XPR Network to A-Chain: Cutover Plan and Cryptographic Guarantees

How the old validator set commits on-chain to stop at block H, how the new chain verifies it before producing H+1, and what each stakeholder does on swap day.

Status: rehearsal PASS at H=170 (five nodes, platform-bound confirmation, H+1 parent-linked, 5/5 head agreement, 4/4 state identical, 10/10 boundary txs exactly-once).
Code is authoritative where this document and the implementation disagree.

1. The swap in one page

The A-Chain runs PulseVM as an Avalanche subnet VM. Before cutover it shadows XPR block by block in real time, replaying every block with full validation. At an agreed height H the XPR validator set commits on-chain to never author past H. The checkpoint (full state at H) is exported, the new chain verifies the commitment against the old schedule's keys, and block H+1 is produced by the A-Chain. XPR is abandoned at H: there is no coexistence, no shared block ids, and the A-Chain id is cryptographically distinct, so XPR signatures cannot replay onto it.

Phase	What happens
Shadow (now -> H-K)	A-Chain replays XPR in real time; operators export snapshots; nothing stops.
Declare (H-K)	Validator set commits to H: declaration digest recorded on-chain; signatures collected.
Freeze (H)	H is the last XPR block; network halts; checkpoint sealed and hashed.
Confirm (H+1 .. H+J)	Quorum chains H, block id, checkpoint and new chain id to the declaration.
Flip (H+1)	Gate verifies the full proof; A-Chain produces H+1 on top of H_ID.

2. Cryptographic guarantees

Every guarantee below is enforced in code at restore time, not asserted in runbooks. All signatures are K1 recoverable signatures over SHA-256 digests, verified by recovery against rostered keys. Reference rehearsal values in parentheses (H=163, K=26).

2.1 Two digests, forced by causality

At H-K the terminal state does not exist, so no one can sign it; at H the pre-commit window is gone. One signature cannot do both jobs, so the ceremony signs two:

- Declaration M_decl = sha256(domain || source_chain_id || H || member_count || per member (name_len || name || compressed_key || weight)), members in byte order. Signable at H-K: everything in it exists.
- Confirmation M_conf = sha256(domain || decl_digest || H || H_ID || checkpoint || new_chain_id). Signable only at H: it needs the terminal block id and the state hash.

2.2 The five enforced rules

Rule	Enforcement
Pair mandatory	No declaration + confirmation pair: no restore. A manifest without it is refused before any state is trusted.
Strict 2/3+1 by weight	Recovered signer weights summed as u128 integers; signed x 3 > total x 2. Exactly-two-thirds fails. Contract and verifier use bit-identical math.
Roster provenance	The set is frozen on-chain from the live producers table, never supplied by a coordinator. The gate re-checks names and keys against the checkpoint state (light-client rule); weights are pinned by the frozen on-chain tally, because vote totals drift every block.
Provable timing	Votes must land before H, proven by on-chain ceremony votes or by a recorded on-chain inclusion (memo / multisig block). A pair with neither fails closed.
Bindings	Checkpoint hash is recomputed from the artifact; confirmation binds H, block id, checkpoint, successor chain id, and the platform chain id. Replays across heights, exports, chains, or platforms cannot verify.

2.3 What forgery would require

Fabricating a cutover means forging secp256k1 signatures from keys holding more than two-thirds of the

scheduled stake, over a digest that simultaneously binds the height, the terminal block id, the exact checkpoint bytes, and the new chain id -- or else subverting the XPR chain itself, which the A-Chain independently replays. There is no single key, no admin override, no coordinator signature anywhere in the path.

3. Stakeholder map: who does what on swap day

Stakeholder	What changes	Action required	When
XPR block producers	Stop at H; two signatures with existing block-signing keys.	Install update; sign declaration; stay online through H; sign confirmation.	H-K and H+1..H+J
A-Chain validators	Start producing at H+1 from the verified checkpoint.	Run PulseVM migration genesis; hold no XPR keys.	H+1 onward
Exchanges	Deposits/withdrawals move to A-Chain rails; new chain id.	Pause XPR flows around H; switch endpoints; publish new deposit addresses.	H-1 .. H+2
Gateways / RPC providers	Serve the new chain; keep XPR archive for history.	Point load balancers at A-Chain; retain one XPR archive node.	H+1
Wallets, dApps, clients	New endpoint + chain id; accounts, keys and balances unchanged.	Ship an update with the new chain id and RPC URL; no key migration.	Before H+1
Explorers / indexers	Index from H+1; pre-H history stays on XPR archives.	Re-point indexers; backfill the H boundary.	H+1

3.1 Exchanges

Balances migrate byte-for-byte inside the checkpoint, so no user action is needed. What changes for an exchange is the rail: a new chain id and new RPC endpoints mean new deposit addresses. Recommended sequence: announce the swap window, pause XPR deposits and withdrawals one hour before H, confirm H+1 finality on the A-Chain plus the offline pair verification receipt, then enable A-Chain flows with fresh addresses. Withdrawals signed on XPR after H are invalid by construction and must not be honored.

3.2 Gateways and RPC providers

Cutover is a DNS/load-balancer event, not a migration: at H+1 traffic moves to A-Chain endpoints. Keep one XPR archive node for pre-H history until indexers backfill. No request signing changes; only the base URL and chain id change.

3.3 Clients, wallets, dApps

The only client-side changes are the chain id and the RPC URL, shipped as a normal app update before H+1. Accounts, permissions and keys are preserved in state, so users do not re-onboard. Contract the new chain id as a constant, never as user input.

4. BP signing coordination: manual path vs automatic update

The only signatures the cutover needs are two per block producer, both made with the block-signing key the producer node already holds in memory for block production. No new keys, no new accounts, no key ceremony. The question is only how the signatures get made.

4.1 Manual path (available today)

- Operator runs the provided ceremony tool, which prints the declaration digest for H.
- Operator signs it with the block-signing key (same key, one CLI command) and submits the vote transaction or memo; repeats after H exists for the confirmation.
- Total: about five minutes of operator work at two touchpoints (before H, after H).
- Downside: needs every producer's operator at the keyboard inside the window; one missed window restarts governance.

4.2 Automatic path (built and live-tested: ship it in the update)

The bullish option, and the one we recommend, now built as xpr_cutover_voter and proven against a live node: bundle it into the scheduled XPR release so producers opt in once, by upgrading. A small watcher daemon polls the ceremony contract's tables, recomputes the declaration digest locally, and -- only on an exact match with the coordinated freeze -- signs it with the block-signing key and submits the vote transaction. Once H exists it signs the confirmation the same way and delivers it to the coordinator.

- BP action reduces to: install the update, drop in the config plus key file, stay online through H. The two touchpoints disappear.
- No new secrets: keys live in owner-only files the daemon refuses to start without -- never on the command line, never in logs; it only signs the two ceremony digests.
- Refuses closed on anything but the exact freeze: wrong H, chain, or digest; non-BP keys; double votes; votes at or after H; an unfinalized ceremony; a staged schedule change (rotation fails closed); coordinator data that disagrees with the local node. Dry-run mode rehearses without signing; print-digest mode emits the exact confirmation bytes for an external agent to sign, so the agent and the voter never disagree.
- Fallback intact: if a producer disables automation, the manual CLI path produces byte-identical votes; mixed fleets verify together.
- Local halt included: with stop_producer armed, the voter pauses its own producer right after delivering the confirmation (Leap producer/pause, verified paused) and keeps enforcing it -- a node restart clears the in-memory pause, so one-shot pausing would reopen the gap. A contract cannot stop block production, only the producer can. Off by default; a pause that will not verify is retried in-loop and logged loudly, and the loop polls every second near H so it lands promptly.

Why this passes governance: the automation cannot sign anything a manual vote could not -- same key, same digests, same window, same on-chain checks -- it only removes the keyboard. The gate verifies signatures, never how they were produced.

4.3 Coordination checklist for BPs

Step	Owner	Deadline
Upgrade to the cutover release; --preflight must print GO.	Each BP operator	H-K-7 days
Confirm dry-run vote matches the published declaration digest.	Each BP operator	H-K-1 day
Declaration window opens; auto (or manual) votes land.	Voter / operator	H-K .. H-1
Keep producing through H; do not stop early.	Each BP operator	H
Confirmation auto-signed once H exists; producer auto-pauses.	Voter / operator	H+1 .. H+J
Repoint any BP-run gateways to A-Chain.	Each BP operator	H+1

Full operator procedure (T-14 publication list, ceremony runbook with the supersede path, post-H audit): docs/xpr-cutover-operations.md.

5. How the A-Chain verifies the commitment and starts producing

Everything above happens on XPR. This is the other half: what a PulseVM node checks at startup before it trusts the imported H-state, and how block production starts at H+1. Both run before any A-Chain block exists, so a bad manifest boots nothing.

5.1 The restore gate (startup, before any block)

Controller::initialize with a migration genesis requires the checkpoint and the manifest together and refuses startup unless the genesis hash matches the manifest. It then runs verify_migration_halt_pair over the two-step pair and cross-checks the declared set against the checkpoint's own producers table. Any failure is a genesis error: the node does not boot.

- Pair mandatory: a manifest with no halt declaration restores nothing; roster-less commitments are rejected everywhere.
- Declaration: version and H must match; every signature must recover to a declared-set key over the declaration digest; the signers' weight must clear strict 2/3+1 in integer math (signed*3 > total*2 over u128, so exactly two-thirds fails).
- Set equality, the light-client rule: the declared set must equal the active XPR schedule read from the checkpoint's own producers table -- same producers, same keys; weights pinned by the frozen tally. The exporter cannot invent members or inflate weights; a rotation between freeze and H fails closed and restarts the ceremony.
- Contract replay: the captured ceremony singleton must be finalized on the same digest; every vote must be a rostered schedule key with its vote block strictly before H, so all timing is XPR consensus, never an asserted timestamp; the recorded tally must equal the recomputed signer weight; the schedule version must be unanimous.
- Confirmation: chains the verified declaration digest to the terminal block id, the checkpoint hash being imported, this chain's id, and the platform chain id it was issued against; the same set's quorum signs that digest. Inclusion must postdate H (it references H_ID) and meet the H+J deadline.
- Chain separation: the successor id is derived from the migration genesis committed to the checkpoint hash, so XPR signatures cannot replay onto the A-Chain.

5.2 H+1 is a start, not a resume

- After a passing restore, the node's last accepted block is the XPR halt block H itself, so the first block it can build is exactly H+1, parent-linked to H_ID -- enforced by construction, asserted by SHiP and every consumer. No gap to fill, no overlap to resolve.
- H+1 is produced and signed by the A-Chain's own genesis schedule (fresh keys from the key ceremony), under the new chain id. XPR producers do not sign A-Chain blocks; their commitment ended at H, proven by the gate above.
- H is the last height the A-Chain will ever honor; post-H XPR activity is never honored on the new chain.
- Boot threshold: no public APIs until two-thirds of the scheduled stake is online and agreeing on the head block id; getInfo then reports the migration chain id with H as base.

6. Verification status

- Restore gate: pair mandatory; roster-less commitments rejected; strict 2/3+1 integer quorum; roster cross-checked against checkpoint state; timing provable via ceremony or recorded inclusion.
- Tests: 317 core + 109 database + 6 verifier tests green, including rejection of pair-less manifests, short quorums, tampered signatures, late votes and inflated tallies; contract fixture 14/14.
- Full rehearsal PASS (five nodes, H=170): offline pair verification green with platform binding, flip gate opened, H+1 parent-linked, 5/5 head agreement, 4/4 accounts identical, 10/10 boundary transactions

exactly-once; a wrong-platform confirmation was refused live before the corrected one opened the gate.

6.1 Known limits

- A schedule rotation between freeze and H fails the ceremony closed by design; freeze close to H and keep the window tight.
- A failed ceremony is a new governance decision, not an automatic retry on the same H.
- Pre-H history stays on XPR archives; the new chain has no transaction lookup index until one is built.